

Module 5 Report: Software Assurance and Secure SQL

Mike Sasso (JHED: msasso1)

JHU EN.605.256 Modern Software Concepts in Python

October 4, 2026

Repository: [git@github.com:sassom2112/jhu_software_concepts.git](https://github.com/sassom2112/jhu_software_concepts) (folder module_5/)

1. Overview

For Module 5 I took the Grad Café web app from Module 4 and hardened it. I brought Pylint to 10.00/10 with its default settings, rebuilt every SQL query with psycopg's sql module, gave every SELECT a LIMIT and clamped every requested limit to 1-100, moved the database credentials into environment variables, and gave the web app its own database role that can only read and add rows. I also made the project installable with pip and uv, drew its dependency graph, scanned it with Snyk, and added a GitHub Actions workflow that checks all of this on every push. The test suite grew from 261 to 331 tests and still covers 100% of module_5/src.

2. Installing and running the project

The project installs the same way with pip or with uv. requirements.txt pins all 32 packages (the app, the tests, Pylint, pydeps and everything they depend on) to exact versions, and setup.py then installs the project itself in editable mode. Python 3.11 is required; the folder's .python-version file pins 3.11.13, and both pyenv and uv read it.

Option A: pip and venv

```
git clone git@github.com:sassom2112/jhu_software_concepts.git
cd jhu_software_concepts/module_5
python3.11 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
pip install -e .
pip check
```

Option B: uv

```
git clone git@github.com:sassom2112/jhu_software_concepts.git
cd jhu_software_concepts/module_5
uv venv .venv # reads .python-version (3.11.13)
source .venv/bin/activate
uv pip sync requirements.txt # makes the environment match the file exactly
uv pip install -e .
uv pip check # a uv venv has no pip of its own
```

uv pip sync also removes anything that is not in requirements.txt, so the project is installed after it. Only the editable install (-e) is supported: the command-line tools find their data files in module_5/, next to src/, and a plain pip install . would copy the modules away from them.

Database, role and app

PostgreSQL 17 runs in Docker. The table owner, gradcafe, loads the data and creates the app's role; the web app then logs in as that role:

```
DATABASE_URL=postgresql://gradcafe@localhost:5432/gradcafe python src/load_data.py
cp .env.example .env                    # then replace each change-me
source .env                             # sets DB_* and APP_DB_*
DATABASE_URL=postgresql://gradcafe@localhost:5432/gradcafe python src/db_roles.py
python src/run.py                       # http://127.0.0.1:8080, logged in as gradcafe_app
```

I tested both install methods from a fresh clone of the GitHub repository:

```
username@fardis:~/fresh_pip/jhu_software_concepts/module_5$ python3.11 -m venv .venv && source .venv/bin/activate && pip install -q --disable-pip-version-check -r requirements.txt && pip install -q --disable-pip-version-check -e . &&
pip check && pip show Werkzeug | grep Version && pylint src | tail -2 && gradcafe-load --help | head -1
No broken requirements found.
Version: 3.1.9
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)

usage: gradcafe-load [-h] [--file FILE] [--reset]
(.venv) username@fardis:~/fresh_pip/jhu_software_concepts/module_5$
```

Figure 1. Fresh clone installed with pip: pip check is clean, Werkzeug is 3.1.9, Pylint scores 10.00/10 and the gradcafe-load command works.

```
(.venv) username@fardis:~/jhu_software_concepts/module_5$ rm -rf ~/fresh_uv && mkdir ~/fresh_uv && cd ~/fresh_uv
(.venv) username@fardis:~/fresh_uv$ git clone https://github.com/sasson2112/jhu_software_concepts.git && cd jhu_software_concepts/module_5
Cloning into 'jhu_software_concepts'...
remote: Enumerating objects: 599, done.
remote: Counting objects: 100% (599/599), done.
remote: Compressing objects: 100% (353/353), done.
remote: Total 599 (delta 289), reused 531 (delta 224), pack-reused 0 (from 0)
Receiving objects: 100% (599/599), 12.24 MiB | 9.46 MiB/s, done.
Resolving deltas: 100% (289/289), done.
Using Python 3.11.13
Creating virtual environment at: .venv
Activate with: source .venv/bin/activate
Resolved 32 packages in 240ms
Installed 32 packages in 29ms
+ astroid==4.0.4
+ beautifulsoup4==4.15.0
+ blinker==1.0.0
+ click==8.5.0
+ coverage==7.16.1
+ dill==0.4.1
+ flask==3.1.3
+ greenlet==3.5.6
+ iniconfig==2.3.0
+ isort==5.0.2
+ itsdangerous==2.2.0
+ Jinja2==3.1.6
+ lxml==6.1.3
+ markupsafe==3.0.3
+ mccabe==0.7.0
+ mypy-extensions==1.1.0
+ packaging==26.3
+ platformdirs==4.12.1
+ pluggy==1.6.0
+ pycodng==3.3.6
+ pycodng-binary==3.3.6
+ pydeps==3.0.8
+ pygments==2.21.0
+ pylint==4.0.9
+ pytest==9.1.1
+ pytest-cov==7.1.0
+ soupsieve==2.9.2
+ sqlalchemy==2.0.54
+ stdlib-list==0.12.0
+ tomlkit==0.15.1
+ typing-extensions==4.16.0
+ Werkzeug==3.1.3
Resolved 16 packages in 774ms
Built gradcafe-analysis @ file:///home/username/fresh_uv/jhu_software_concepts/module_5
Prepared 1 package in 15ms
Installed 1 package in 4ms
+ gradcafe-analysis==0.5.0 (from file:///home/username/fresh_uv/jhu_software_concepts/module_5)
Checked 33 packages in 0.73ms
All installed packages are compatible
(.venv) username@fardis:~/fresh_uv/jhu_software_concepts/module_5$ pylint src && gradcafe-load --help | head -1
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)

usage: gradcafe-load [-h] [--file FILE] [--reset]
(.venv) username@fardis:~/fresh_uv/jhu_software_concepts/module_5$ deactivate; rm -rf ~/fresh_uv; cd ~/jhu_software_concepts/module_5 && source .venv/bin/activate
(.venv) username@fardis:~/jhu_software_concepts/module_5$
```

Figure 2. Fresh clone installed with uv: uv venv uses Python 3.11.13, uv pip sync installs the 32 pinned packages, and Pylint scores 10.00/10. (Taken before the Werkzeug upgrade in section 8.)

3. Pylint 10.00/10

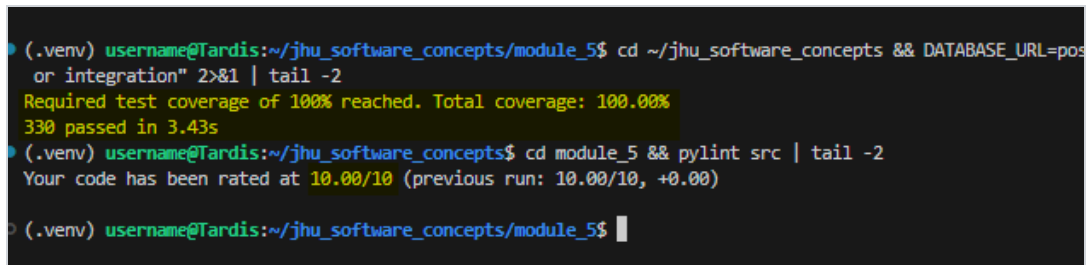
```
cd module_5
pylint src          # Your code has been rated at 10.00/10
```

I used Pylint's default settings, with no .pylintrc file, and changed the code instead of switching checks off. CI runs the same check as `pylint src --fail-under=10`.

- `scrape.py` was too large, so I split it into `site_urls.py` (URL rules), `robots_rules.py` (robots.txt), `scrape_state.py` (resume state) and `jsonio.py` (JSON files).
- I removed a circular import and moved shared code into `db_config.run_with_connection()` and `analysis_common.print_answers()`.
- The Flask routes now catch a short list of expected errors instead of every Exception.

Pylint is switched off by four comments, each saying why it is wrong there. The not-callable one covers all of `orm_queries.py`; the other three are on single lines:

Check	Where	Why it is a false positive
not-callable	<code>orm_queries.py</code> (whole module)	SQLAlchemy generates <code>func.count()</code> and similar at run time
too-few-public-methods (2)	<code>models.Base</code> , <code>models.Applicant</code>	ORM classes carry SQLAlchemy's mapping; they need no public methods
invalid-name	<code>models.SessionLocal</code>	the usual SQLAlchemy name for a session factory



```
(.venv) username@Tardis:~/jhu_software_concepts/module_5$ cd ~/jhu_software_concepts && DATABASE_URL=postgres://postgres:postgres@localhost:5432/postgres && python3 -m unittest discover -s . -p '*_test.py' --verbose
or integration" 2>&1 | tail -2
Required test coverage of 100% reached. Total coverage: 100.00%
330 passed in 3.43s
(.venv) username@Tardis:~/jhu_software_concepts$ cd module_5 && pylint src | tail -2
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)
(.venv) username@Tardis:~/jhu_software_concepts/module_5$
```

Figure 3. `pylint src` scores 10.00/10; the same screenshot shows the test suite at 100% coverage (330 tests at the time; 331 after the test added in section 9).

4. SQL injection defenses

Every query in `module_5` is now built with `psycopg2`'s `sql` module. Values are always sent as bound parameters, so text such as `' OR '1'='1` is only ever compared as a string and never runs as SQL. Table and column names cannot be bound, so they come from allow-lists in the code and are quoted with `sql.Identifier`.

The only place where text typed by a user reaches SQL is the new search API, `GET /api/applicants`. `parse_search_args()` checks every parameter against allow-lists (filter names, sort columns, asc or desc) and refuses values over 100 characters; `build_search_query()` composes the statement; and `search_applicants()` runs it on a read-only connection. Anything invalid gets a 400 with a fixed message, and a database error gets a 500 with a fixed message, so no database detail reaches the caller. This is the function that builds the statement, copied from `src/applicant_search.py`:

```

def build_search_query(request: SearchRequest) -> tuple[sql.Composed, dict[str, object]]:
    """Compose the SELECT for a validated request; returns (statement, params). No I/O."""
    conditions = []
    params: dict[str, object] = {}
    for name, value in request.filters.items():
        column = sql.Identifier(name)
        if name in CONTAINS_FILTERS:
            conditions.append(
                sql.SQL("{} ILIKE {} ESCAPE '\\').format(column, sql.Placeholder(name))
            )
            params[name] = f"%{_escape_like(value)}%"
        else:
            conditions.append(
                sql.SQL("LOWER(TRIM({})) = LOWER({})).format(column, sql.Placeholder(name))
            )
            params[name] = value
    where = sql.SQL(" WHERE ") + sql.SQL(" AND ").join(conditions) if conditions else sql.SQL("")

    statement = sql.SQL(
        "SELECT {columns} FROM {table}{where} "
        "ORDER BY {sort} {direction} NULLS LAST, {key} DESC LIMIT {limit}"
    ).format(
        columns=sql.SQL(", ").join(sql.Identifier(c) for c in RESULT_COLUMNS),
        table=sql.Identifier(TABLE_NAME),
        where=where,
        sort=sql.Identifier(request.sort),
        direction=_DIRECTIONS[request.order],
        key=sql.Identifier("p_id"),
        limit=sql.Placeholder("limit"),
    )
    params["limit"] = clamp_limit(request.limit)
    return statement, params

```

Every row limit that comes from outside the code goes through `clamp_limit()` in `src/query_limits.py` first:

```

def clamp_limit(value: object, default: int = DEFAULT_LIMIT) -> int:
    """Turn a requested row limit into an int between 1 and MAX_LIMIT.

    ``None`` or a blank string means "use the default". Anything that is not a
    plain whole number - ``"abc"`` , ``"5.5"`` , ``"1e9"`` , ``"10; DROP TABLE x"`` -
    raises LimitError instead of being guessed at. Whole numbers outside
    1..MAX_LIMIT are pulled to the nearest end, so ``"0"`` gives 1 and
    ``"1000000"`` gives MAX_LIMIT.
    """
    if value is None or (isinstance(value, str) and not value.strip()):
        return default
    if isinstance(value, bool) or not isinstance(value, (int, str)):
        raise LimitError("limit must be a whole number")
    text = str(value).strip()
    if not _WHOLE_NUMBER.fullmatch(text):
        raise LimitError("limit must be a whole number")
    if len(text.lstrip("+")) > _MAX_DIGITS:
        return 1 if text.startswith("-") else MAX_LIMIT
    return max(1, min(MAX_LIMIT, int(text)))

```

Requirements met

Requirement	How	Where
LIMIT on every query	Every SELECT has a LIMIT, and the ORM queries use .limit(). The loader's INSERT ... SELECT is capped at its batch size; DDL, GRANT and COPY take no LIMIT	query_data.py (Q1-Q11), load_data.py, applicant_search.py, webapp/services.py, db_roles.py, build_query_results.py, orm_queries.py
Limit clamped to 1-100	clamp_limit(): 0 becomes 1, 1000000 becomes 100; anything that is not a whole number is refused	query_limits.py
Statement construction separated from execution	build_search_query() returns (statement, params) and does no I/O; search_applicants() only executes. The other modules keep their composed statements as module-level constants	applicant_search.py, query_data.py, load_data.py
Safe composition and parameters	sql.SQL templates, sql.Identifier for names, sql.Placeholder %(name)s for values. SQL text is never built from Python strings with f-strings, + or str.format(); the one + in applicant_search.py joins two sql objects, and the one f-string there builds a LIKE value that is then bound. The role password is the only value written into SQL text, because CREATE ROLE cannot bind it; sql.Literal quotes and escapes it	all of src/
Malicious input handled	38 tests send ' OR '1'='1, '; DROP TABLE applicants; --, a UNION SELECT on pg_shadow, bad sort columns and directions, LIKE wildcards and limits such as 10; DROP TABLE applicants; nothing extra comes back and every row is still there	tests/test_sql_injection.py

5. Least-privilege database user

Before Module 5 the web app logged in as gradcafe, a superuser that owns the table and could drop it. Now the app logs in as gradcafe_app, which src/db_roles.py creates. The app only reads rows (the analysis page, the search API, the ids that Pull Data skips) and adds rows (Pull Data), so that is all the role may do.

Permission	Granted	Why
LOGIN	yes	the app has to connect
CONNECT on database gradcafe, USAGE on schema public	yes	needed to reach the table
SELECT on applicants	yes	analysis page, search API, the ids Pull Data skips
INSERT on applicants	yes	Pull Data adds rows; ON CONFLICT DO NOTHING needs no UPDATE
UPDATE, DELETE, TRUNCATE	no	the app never changes or removes rows
CREATE in the schema, ALTER, DROP, owning the table	no	only the administrator's load_data.py creates the table; Pull Data no longer tries
SUPERUSER, CREATEDB, CREATEROLE, REPLICATION, BYPASSRLS	no	nothing in the app needs them
INHERIT	no (NOINHERIT)	a role granted to it later cannot quietly widen what it can do
CONNECTION LIMIT 10, statement_timeout 30s	limits	a misbehaving app cannot tie up the server

These are the statements db_roles.py runs, in one transaction (the real password is never printed):

```
CREATE ROLE "gradcafe_app" WITH LOGIN NOSUPERUSER NOCREATEDB NOCREATEROLE NOINHERIT NOREPLICATION
NOBYPASSRLS CONNECTION LIMIT 10 PASSWORD '...';
ALTER ROLE "gradcafe_app" SET statement_timeout = '30s';
REVOKE ALL PRIVILEGES ON TABLE "applicants" FROM "gradcafe_app";
GRANT CONNECT ON DATABASE "gradcafe" TO "gradcafe_app";
GRANT USAGE ON SCHEMA "public" TO "gradcafe_app";
GRANT SELECT, INSERT ON TABLE "applicants" TO "gradcafe_app";
```

The staging table that Pull Data uses is a temporary table, which every role may create by default, so it needs no extra grant. Credentials come only from environment variables (DB_HOST, DB_PORT, DB_NAME, DB_USER, DB_PASSWORD) or ~/.pgpass; .env.example lists the names with placeholder values, and .env is git-ignored. tests/test_db_hardening.py logs in as a throwaway copy of the role and checks that UPDATE, DELETE, TRUNCATE, DROP TABLE, ALTER TABLE, CREATE TABLE and CREATE ROLE are all refused.

```
(.env) username@Tardis:~/jhu_software_concepts/module_5$ source .env
(.env) username@Tardis:~/jhu_software_concepts/module_5$ DATABASE_URL=postgresql://gradcafe@localhost:5432/gradcafe python src/db_roles.py
Role gradcafe_app is ready (provisioned through gradcafe@localhost:5432/gradcafe; its password is set but never shown)
rolsuper      False
rolcreatedb   False
rolcreaterole False
rolinherit    False
rolreplication False
rolbypassrls  False
rolcanlogin   True
rolconnlimit  10
rolconfig     ['statement_timeout=30s']
privileges on applicants: INSERT, SELECT
(.env) username@Tardis:~/jhu_software_concepts/module_5$
```

Figure 4. Creating the role: every attribute is off except LOGIN, and the role's only privileges on applicants are INSERT and SELECT.

```
gradcafe-# \du gradcafe_app
List of roles
Role name | Attributes
-----+-----
gradcafe_app | No inheritance+
| 10 connections

gradcafe-# \drds gradcafe_app
List of settings
Role | Database | Settings
-----+-----+-----
gradcafe_app | | statement_timeout=30s
(1 row)

gradcafe-# \dp applicants
Access privileges
Schema | Name | Type | Access privileges | Column privileges | Policies
-----+-----+-----+-----+-----+-----
public | applicants | table | gradcafe=arwdDxtm/gradcafe+
| | | gradcafe_app=ar/gradcafe | |
(1 row)

gradcafe-#
```

Figure 5. The same role in *psql*: no inheritance, 10 connections, *statement_timeout*=30s, and *gradcafe_app*=ar/gradcafe (*a* = *INSERT*, *r* = *SELECT*).

6. Dependency graph

```
cd module_5
pydeps src/run.py --noshow --max-bacon 0 --max-module-depth 1 -T svg -o dependency.svg
```

The app has no `app.py`; it starts from `src/run.py`. `--max-bacon 0` follows every import (the default of 2 showed only 7 modules), and `--max-module-depth 1` draws each library as one box instead of hundreds of its internal modules.

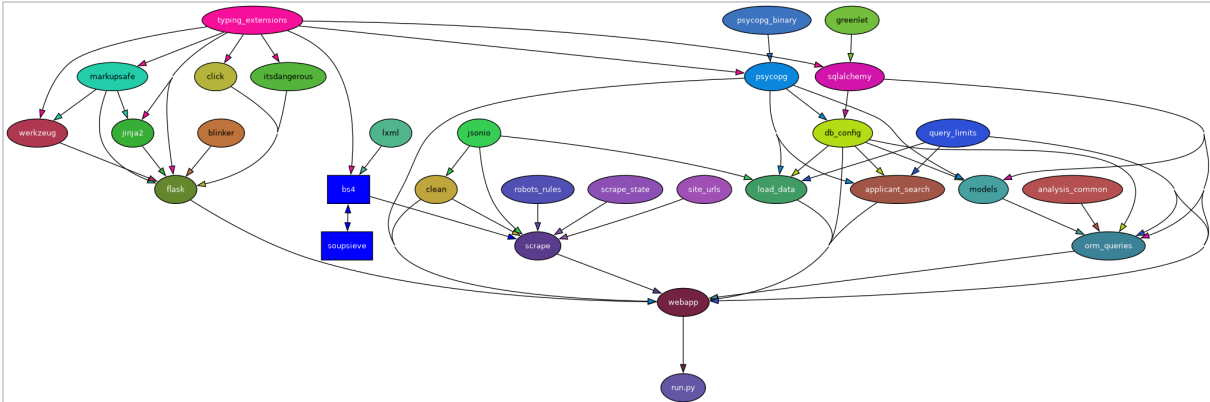


Figure 6. *dependency.svg*, rendered as an image for this report.

src/run.py is the entry point: it imports only webapp, the Flask package with the routes and services, and every arrow in the graph points from the imported module to the module that imports it. webapp is built on Flask, which brings in Werkzeug for HTTP handling and the development server, Jinja2 and MarkupSafe for HTML templates with automatic escaping, Click for the flask command, ItsDangerous for signed cookies and Blinker for signals. For its data, webapp calls scrape (Pull Data), clean (scraped text into table fields), load_data (inserting rows), and orm_queries and applicant_search (the analysis page and the search API). scrape parses Grad Café pages with BeautifulSoup, which uses the lxml parser and soupsieve for CSS selectors, and its small

helpers jsonio, robots_rules, scrape_state and site_urls came out of the Pylint refactor. db_config is the one module every database path goes through, so the connection settings from the environment live in one place; psycopg (with its compiled psycopg_binary driver) runs the composed SQL, and SQLAlchemy (with greenlet) runs the ORM in models and orm_queries. query_limits is imported by load_data, applicant_search, orm_queries and webapp, which is how one 1-100 LIMIT rule covers every query path. The command-line tools query_data, db_roles and build_query_results are not in the graph because the web app never imports them.

7. Why packaging matters

setup.py turns module_5 from a folder of scripts into a project that pip and uv can install. After pip install -e ., every module imports the same way from any folder, so the tests, the command-line tools and CI no longer depend on where they are started, and two console commands, gradcafe-load and gradcafe-app-role, work from anywhere. The editable install links to src/ instead of copying it, so a change takes effect immediately without reinstalling, which removes a common cause of "works on my machine" problems.

setup.py names only what the app needs at run time, with the oldest version of each package it works with. I found those minimums by installing older releases and running the tests: psycopg needs 3.2.4 and SQLAlchemy needs 2.0.18. requirements.txt is the other half: the exact set of 32 versions that was tested, which pip install -r and uv pip sync rebuild on any machine. uv can also read the dependencies straight from setup.py with uv pip compile setup.py.

8. Snyk dependency scan

```
cd module_5
source .venv/bin/activate      # Snyk reads the installed packages
snyk test --file=requirements.txt --package-manager=pip
```

Snyk tested all 32 dependencies and found one Medium issue: Werkzeug 3.1.8, "Improper Handling of Windows Device Names" (SNYK-PYTHON-WERKZEUG-20305246). On Windows, Werkzeug's safe_join() let device names such as NUL: through, so a request for /static/NUL: could hang a server worker. The app serves /static, so the code is reachable, but the check only exists on Windows, and this app runs on Linux, WSL, Docker and the CI runners. I upgraded anyway: one line in requirements.txt, Werkzeug==3.1.9. All 331 tests and Pylint 10.00/10 still pass, and the scan now finds no vulnerable paths.

```
(.venv) username@Tardis:~/jhu_software_concepts/module_5$ cd ~/jhu_software_concepts/module_5 && source .venv/bin/activate
(.venv) username@Tardis:~/jhu_software_concepts/module_5$ snyk test --file=requirements.txt --package-manager=pip

Testing /home/username/jhu_software_concepts/module_5...

Tested 32 dependencies for known issues, found 1 issue, 2 vulnerable paths.

Issues to fix by upgrading dependencies:

Upgrade werkzeug@3.1.8 to werkzeug@3.1.9 to fix
X Improper Handling of Windows Device Names (new) [Medium Severity][https://security.snyk.io/vuln/SNYK-PYTHON-WERKZEUG-20305246] in werkzeug@3.1.8
  introduced by werkzeug@3.1.8 and 1 other path(s)

Organization:      sassom2112
Package manager:   pip
Target file:       requirements.txt
Project name:      module_5
Open source:       no
Project path:      /home/username/jhu_software_concepts/module_5
Licenses:          enabled

Tip: Detected multiple supported manifests (2), use --all-projects to scan all of them at once.
```

Figure 7. snyk test before the fix: one Medium issue in Werkzeug 3.1.8.

```
Requirement already satisfied: stdlib-list==0.12.0 in ~/.venv/lib/python3.11/site-packages (from -r requirements.txt (line 46)) (0.12.0)
Using cached werkzeug-3.1.9-py3-none-any.whl (228 kB)
Installing collected packages: Werkzeug
  Attempting uninstall: Werkzeug
    Found existing installation: Werkzeug 3.1.8
    Uninstalling Werkzeug-3.1.8:
      Successfully uninstalled Werkzeug-3.1.8
  Successfully installed Werkzeug-3.1.9

[notice] A new release of pip is available: 24.0 -> 26.2.1
[notice] To update, run: pip install --upgrade pip
(.venv) username@Tardis:~/jhu_software_concepts/module_5$ snyk test --file=requirements.txt --package-manager=pip

Testing /home/username/jhu_software_concepts/module_5...

Organization:      sassom2112
Package manager:   pip
Target file:       requirements.txt
Project name:      module_5
Open source:       no
Project path:      /home/username/jhu_software_concepts/module_5
Licenses:          enabled

✓ Tested 32 dependencies for known issues, no vulnerable paths found.

Tip: Detected multiple supported manifests (2), use --all-projects to scan all of them at once.

Next steps:
- Run `snyk monitor` to be notified about new related vulnerabilities.
- Run `snyk test` as part of your CI/test.
```

Figure 8. After pinning Werkzeug 3.1.9: no vulnerable paths (snyk-analysis.png).

9. Extra credit: Snyk Code

snyk code test reads my own code and follows data from where it enters to where it is used. The first run reported 5 issues (0 High, 1 Medium, 4 Low). None was exploitable:

Finding	Verdict	Reason
Medium: SQL injection, src/db_roles.py:183	false positive	The value is the database's own name, read from the server, and sql.Identifier quotes it. A new test shows that x"; DROP TABLE applicants; -- stays one quoted name. GRANT cannot take a bound parameter for a name, which is exactly what sql.Identifier is for.
Low: hardcoded password, tests/test_db_hardening.py:79	false positive, changed anyway	It was a fake value in a test. It is now secrets.token_urlsafe(16), a new random value each run.
Low: path traversal (3), llm_hosting/app.py:605, 612, 623	false positive	Command-line arguments of a local tool: the person running it already has those files, and _confine_path() keeps the paths inside the module folder.

The second run reports 4 issues (0 High, 1 Medium, 3 Low), all explained above. Snyk Code also missed two real problems that I found by reading the code, and I fixed both. Pull Data returned the raw exception text to the browser, which could include the database host and user, and with a malformed DATABASE_URL even the password; it now returns "the pull could not be completed" and logs only the exception's class name. The optional LLM server in llm_hosting listened on every network interface with no login; it now listens on 127.0.0.1 unless FLASK_HOST says otherwise.

```
(.venv) username@ardis:~/jhu_software_concepts/module_5$ cd ~/jhu_software_concepts && DATABASE_URL=postgresql://gradcafe@localhost:5432/gradcafe_test module_5/.venv/bin/python -m pytest module_5 -a "web or buttons on analysis or db or integration" 2>&1 | tail -2
Required test coverage of 100% reached. Total coverage: 100.00%
331 passed in 4.55s
(.venv) username@ardis:~/jhu_software_concepts$ cd module_5 && pylint src | tail -2 && snyk code test
Your code has been rated at 10.00/10 (previous run: 10.00/10, +0.00)

Testing /home/username/jhu_software_concepts/module_5 ...

Open Issues

X [LOW] Path Traversal
Finding ID: 05235632-6731-49b2-9af0-702767a23287
Path: llm_hosting/app.py, line 605
Info: Unsanitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to read arbitrary files.

X [LOW] Path Traversal
Finding ID: 3ea67979-7af1-43c7-8fa3-fc39601f6fd5
Path: llm_hosting/app.py, line 612
Info: Unsanitized input from a command line argument flows into open, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

X [LOW] Path Traversal
Finding ID: 046c5501-e315-4505-b215-dbf8acc515bd
Path: llm_hosting/app.py, line 623
Info: Unsanitized input from a command line argument flows into json.dump, where it is used as a path. This may result in a Path Traversal vulnerability and allow an attacker to write arbitrary files.

X [MEDIUM] SQL Injection
Finding ID: f8eaf05d-2f1d-4f22-8140-4beec723980
Path: src/db_roles.py, line 183
Info: Unsanitized input from a database flows into execute, where it is used in an SQL query. This may result in an SQL Injection vulnerability.

Test Summary
Organization: sassom2112
Test type: Static code analysis
Project path: /home/username/jhu_software_concepts/module_5
Total issues: 4
Ignored issues: 0 [ 0 HIGH 0 MEDIUM 0 LOW ]
Open issues: 4 [ 0 HIGH 1 MEDIUM 3 LOW ]

Tip
```

Figure 9. 331 tests pass, Pylint 10.00/10, and snyk code test reports 0 High, 1 Medium and 3 Low (snyk-code-analysis.png).

10. Continuous integration

.github/workflows/ci.yml runs four jobs on every push or pull request that changes module_5/. module_5/.github/workflows/ci.yml is an identical copy, and the pytest job fails if the two differ.

Job	What it runs	Fails when
Pylint	pylint src --fail-under=10	the score is below 10.00
Dependency graph	installs Graphviz, regenerates dependency.svg with pydeps, uploads it as an artifact	the committed dependency.svg is missing, or its modules and imports differ from the regenerated graph
Snyk	snyk test, then snyk code test, with the SNYK_TOKEN repository secret	any known vulnerability in the pinned packages, any High code finding, or a missing token
Pytest	PostgreSQL 17 with real password checks, then all 331 tests from the repository root	any test fails, or coverage is below 100%

The graph is compared by its modules and imports rather than byte for byte, because a different Graphviz version draws the same graph with a different layout. snyk code test lists every finding in the log but fails only on High, because the Medium and Low findings are the false positives in section 9. The first run passed in 47 seconds.

```

● username@Tardis:~/jhu_software_concepts$ gh run watch
? Select a workflow run * module_5: GitHub Actions ci.yml with four jobs (Pylint --fail-under=10, ci [master] 2s ago
✓ master ci · 36994430841
Triggered via push about 16 minutes ago

JOBS
✓ Pytest (100% coverage required) in 32s (ID 110797781122)
  ✓ Set up job
  ✓ Initialize containers
  ✓ Run actions/checkout@v7
  ✓ The workflow copy under module_5 matches this file
  ✓ Run actions/setup-python@v7
  ✓ Install the pinned environment and the project
  ✓ Run the test suite from the repository root (pytest.ini measures module_5/src)
  ✓ Post Run actions/setup-python@v7
  ✓ Post Run actions/checkout@v7
  ✓ Stop containers
  ✓ Complete job
✓ Snyk (dependencies and code) in 42s (ID 110797781350)
  ✓ Set up job
  ✓ Run actions/checkout@v7
  - No SNYK_TOKEN secret - skipped on a pull request from a fork, an error otherwise
  ✓ Run actions/setup-python@v7
  ✓ Install the pinned environment (Snyk reads the installed packages)
  ✓ Install the Snyk CLI
  ✓ snyk test - known vulnerabilities in the pinned dependencies (fails on any)
  ✓ snyk code test - static analysis of our code (lists every finding, fails on High)
  ✓ Post Run actions/setup-python@v7
  ✓ Post Run actions/checkout@v7
  ✓ Complete job
✓ Dependency graph (pydeps + Graphviz) in 32s (ID 110797781370)
  ✓ Set up job
  ✓ Run actions/checkout@v7
  ✓ Run actions/setup-python@v7
  ✓ Install Graphviz (pydeps runs its dot program)
  ✓ Install the pinned environment and the project
  ✓ Generate dependency.svg
  ✓ dependency.svg was produced and matches the committed graph
  ✓ Run actions/upload-artifact@v7
  ✓ Post Run actions/setup-python@v7
  ✓ Post Run actions/checkout@v7
  ✓ Complete job
✓ Pylint (10.00/10 required) in 29s (ID 110797781380)
  ✓ Set up job
  ✓ Run actions/checkout@v7
  ✓ Run actions/setup-python@v7
  ✓ Install the pinned environment and the project
  ✓ Pylint, failing below 10
  ✓ Post Run actions/setup-python@v7
  ✓ Post Run actions/checkout@v7
  ✓ Complete job
✓ Run ci (36994430841) completed with 'success'

```

Figure 10. A green run of ci.yml: all four jobs and their steps passed.

11. Limitations

- Pull Data stops after 50 pages and the next click starts again at page 1, so if more than 50 pages of new entries ever pile up, the older part of them is never fetched.
- The busy flag that blocks a second Pull Data belongs to one Python process; a server with several workers would need a shared lock.
- Rows added by Pull Data have empty LLM columns until the optional llm_hosting standardizer runs.
- Only editable installs are supported, because the tools find their data files next to src/.
- The Sphinx documentation was written for Module 4 and carried over; Read the Docs still builds the Module 4 version.